



AI Agents

SIM Data Analytics Club

Lesson Objectives

By the end of this session, subcomms should be able to:

1. **Understand what AI and AI Agents are:** Definitions, types, and architectures.
2. **Grasp the core components of AI Agents:** Models, tools, knowledge, interfaces, guardrails, orchestration, prompts.
3. **Dive deeper into Models & LLMs:** What they are, how they work, their challenges, and how to choose them.
4. **Explore the role of tools and frameworks:** Like LangChain in building agents.
5. **Apply the concepts by coding their own AI Agent:** Using Python using LangChain.



Table of contents

01

The Big Picture

02

Components of Modern AI Agents

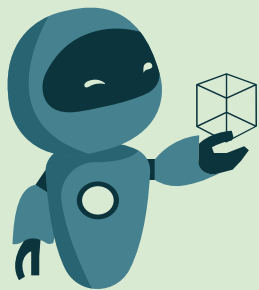
03

Design Process for building an an AI Agent

04

Hands-on: Build Your Own AI Agent

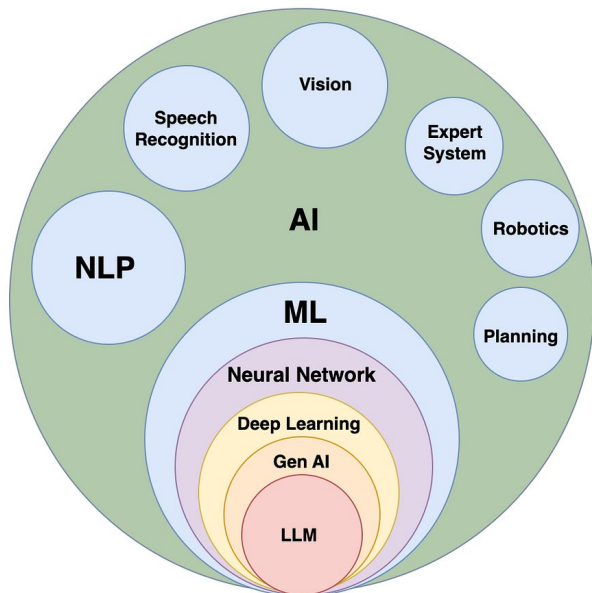




01

The Big Picture

Artificial Intelligence Landscape



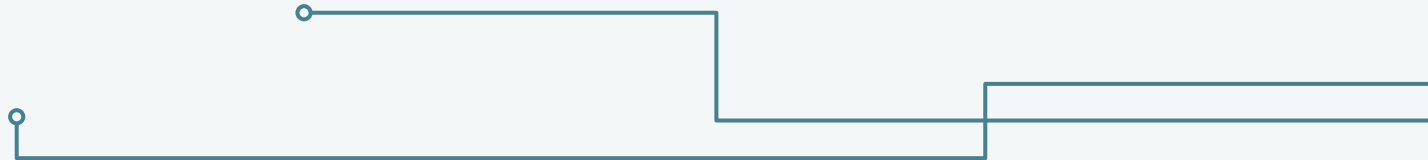
AI is technology that enables computers and machines to **simulate human** learning, comprehension, problem solving, decision making, creativity and autonomy.

What is an AI Agent?



An AI agent is a software-based system that perceives its environment, processes information, makes decisions and takes action to achieve a goal.

AI agents range from basic automation tools to advanced, independent decision-makers.



Evolution of AI Agents



Rule-based systems

- Simple automation scripts
- IF-THEN logic
- No memory, no planning



Goals & utility based agents

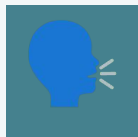
- Internal model of the world.
- Could set goals and optimize.
- Could anticipate and plan ahead.



LLM-powered agentic systems

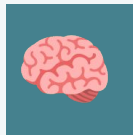
- Natural language understanding
- Chain of reasoning + tool use + memory
- Plan, select tools, learn, and act

Key Characteristics of an AI Agent



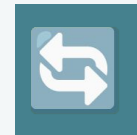
Natural Language Interface

- Understand complex instructions
- Example: "Find articles from the last 3 months about AI safety and summarize key concerns"



Multi-Step Reasoning

- Break down complex tasks into steps
- Example: Research → Filter → Analyze → Synthesize → Present



Tool Use

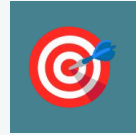
- Can call external functions and APIs
- Example: Uses `web_search` when it needs current info, uses calculator for math

Key Characteristics of an AI Agent



Memory & Context

- Remember conversation history
- Example: "As I mentioned earlier..." - references past messages



Goal-Oriented Autonomy

- Work toward objectives independently
- Example: If search returns no results, tries different query

Types of AI Agent

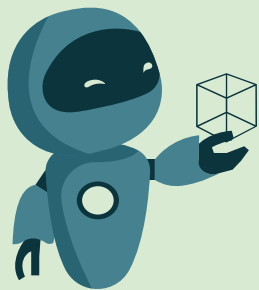
1. **Simple Reflex Agent:** Acts using predefined condition–action rules.
2. **Model-Based Reflex Agent:** Enhance the previous agent by maintaining an internal state or model of the world.
3. **Goal-Based Agent:** Select actions by considering future states relative to explicit goals.
4. **Utility-Based Agent:** Extends previous agent by considering not only whether a goal is met but also how valuable a particular outcome is
5. **Learning Agents:** Unlike other AI agents, which rely on predefined rules or models, learning agents continuously update their behavior based on feedback from the environment. They are useful in applications such as autonomous driving, robotics and virtual assistants that assist human agents in customer support.
6. **Multi-agent systems:** These agents are designed to break down complex problems into smaller, manageable subtasks, making it easier to handle complex problems in real-world scenarios.
7. **Hierarchical agents:** Organize behavior into multiple layers such as strategic, tactical and operational.

Comparison of AI Agents

Agent Type	Main Strength	Limitations	Best For	Example
Simple Reflex Agent	Instant reaction based on fixed rules	No memory or learning; fails in dynamic environments	Fully observable, stable and simple environments	Traffic light timers
Model-Based Reflex Agent	Handles partial observability with internal state	More computational demand; depends on model accuracy	Dynamic or partially observable environments	Robot vacuum cleaners
Goal-Based Agent	Plans ahead to achieve specific objectives	Needs clear goals and planning algorithms	Strategic tasks with defined goals	Logistics route planning
Utility-Based Agent	Balances multiple factors for best outcome	Requires complex utility functions	Multi-criteria decision-making	Financial portfolio management

Comparison of AI Agents

Agent Type	Main Strength	Limitations	Best For	Example
Learning Agent	Improves over time via experience	Needs data and training time	Dynamic environments with changing conditions	AI chatbots
Multi-Agent System (MAS)	Distributed problem-solving with cooperation or competition	Complex interactions; unpredictable behaviors	Decentralized, multi-entity systems	Smart traffic control
Hierarchical Agent	Breaks complex tasks into levels for efficiency	Requires well-defined interfaces between layers	Large-scale, multi-level operations	Drone delivery management



02

Components of a Modern AI Agent

Components of AI Agents



1. Models

The core intelligence for reasoning and decision-making. Usually LLMs.



2. Tools

External systems and APIs that agents use to perform actions in the real world.



3. Knowledge

Systems for storing and retrieving information, including past interactions and learned data.



4. Interfaces

How agents interact with the outside world, such as web apps or APIs.



5. Orchestration

Manages tasks, steps, and workflows to guide the agent's actions.



6. Instruction

Clear instructions and context that guide the agent's focus.



7. Guardrails

Safety mechanisms to ensure proper behavior.



Large Language Models (LLMs)

A **Large Language Model** is a type of deep learning model specifically trained on text data to understand and generate human language.

Three key words:

1. **Large:** Billions of parameters (tunable numbers that store knowledge)
 2. **Language:** Trained on text (books, websites, conversations)
 3. **Model:** A mathematical function (input text → output text)
- 



Large Language Models (LLMs)

Popular models:

- GPT-4 (OpenAI) – 1+ trillion parameters
 - Claude (Anthropic) – ~200 billion parameters
 - Gemini (Google) – multi-modal, 1+ trillion parameters
 - LLaMA (Meta) – open-source, 7B to 70B parameters
- 

What LLMs can do

- ✓ Text generation (essays, stories, code)
- ✓ Question answering ("What's the capital of France?")
- ✓ Translation (English → Spanish)
- ✓ Summarization (10 pages → 3 sentences)
- ✓ Code generation (Python, JavaScript, SQL)
- ✓ Reasoning (logic puzzles, math problems)
- ✓ Conversation (natural dialogue)

What LLMs cannot do

- ✗ Access real-time data (no internet browsing)
- ✗ Perform actions (send emails, book flights, query databases)
- ✗ Remember past conversations (each chat is isolated)
- ✗ Guarantee factual accuracy (they predict plausible text)
- ✗ Update their knowledge (training is a one-time event)



This is where AGENTS come in:

Agent = LLM (brain) + Tools (hands) + Memory + Orchestration



Limitations and Risks

1. **Hallucinations:** LLMs confidently generate false information
 2. **Bias:** They inherit biases from training data
 3. **No true understanding:** They manipulate symbols, not meanings
 4. **Context window limits:** They can't remember everything
 5. **Compute cost:** Training costs millions of dollars
- 

Key Properties of Models

- ✓ 💡 **Capability:** Does it understand complex instructions?
- ✓ ⚡ **Speed:** How fast does it respond?
- ✓ 💰 **Cost:** Token pricing for your use case
- ✓ 🧠 **Context Window:** How much information can it process?
- ✓ 🛠️ **Function Calling:** Can it use tools effectively?

How to access models

API Access

- Pay per use, no training needed
- Access via OpenAI, Anthropic, Google APIs
- Best for: Most applications, fast development
- Cost: ~\$0.002 per 1000 tokens (very cheap for learning)

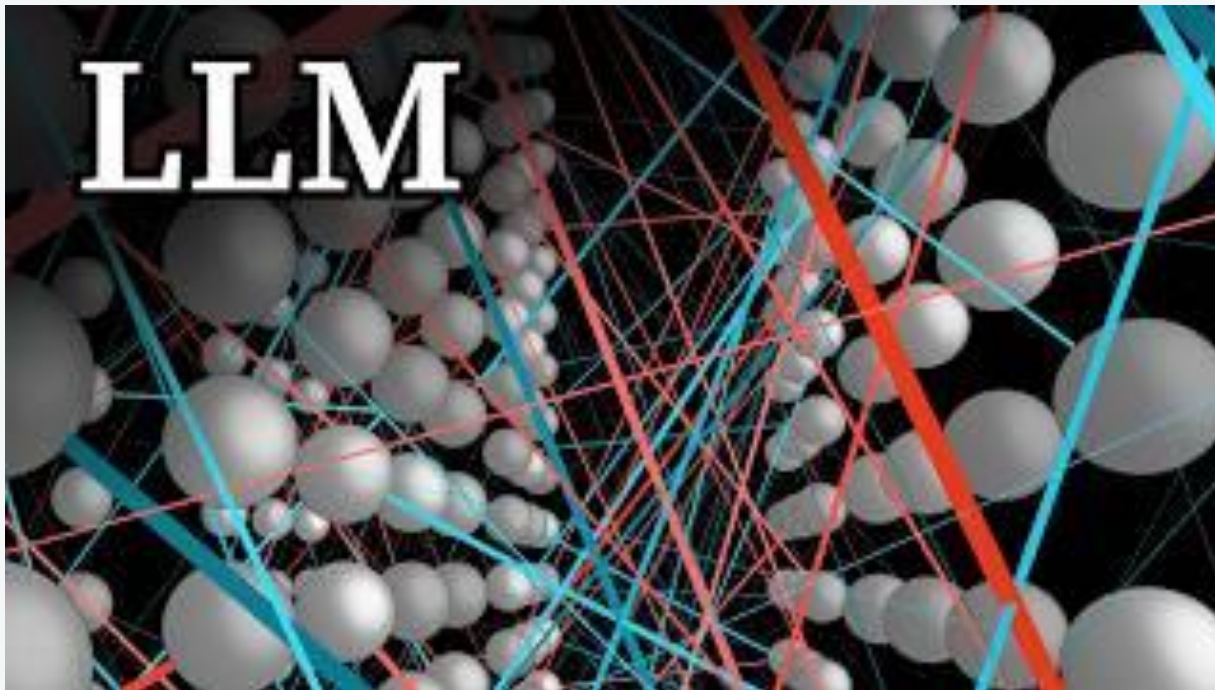
Fine-Tuning

- Customize model on your specific data
- Still use provider's infrastructure
- Best for: Specialized domains, consistent style
- Cost: Training cost + usage cost

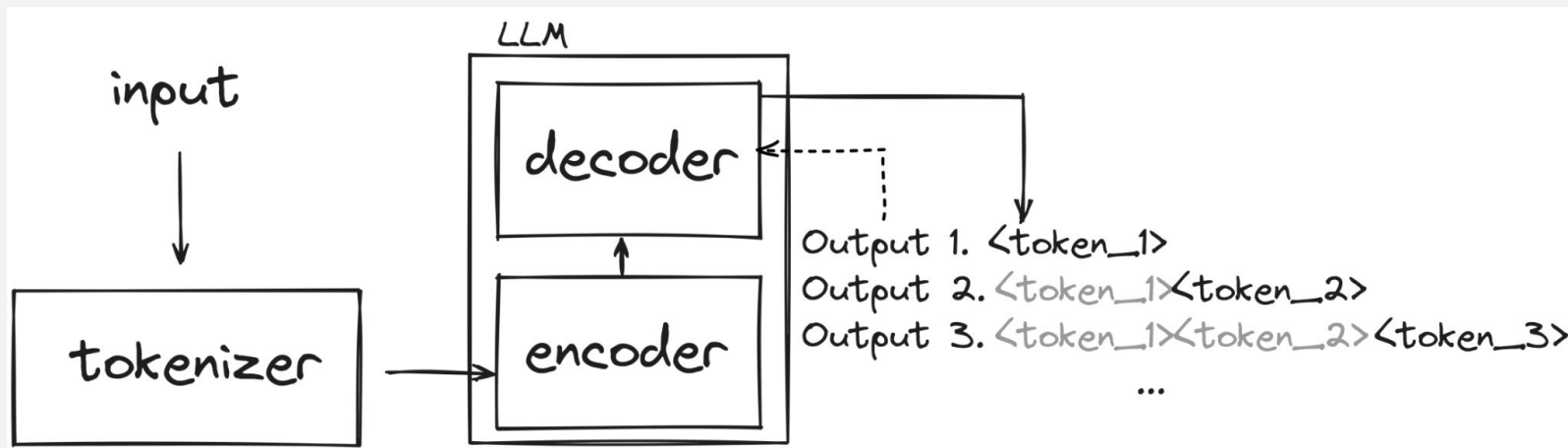
Training from Scratch

- Build your own model
- Requires massive compute and data
- Best for: Research, unique requirements, proprietary needs
- Cost: Millions of dollars

How LLMs work

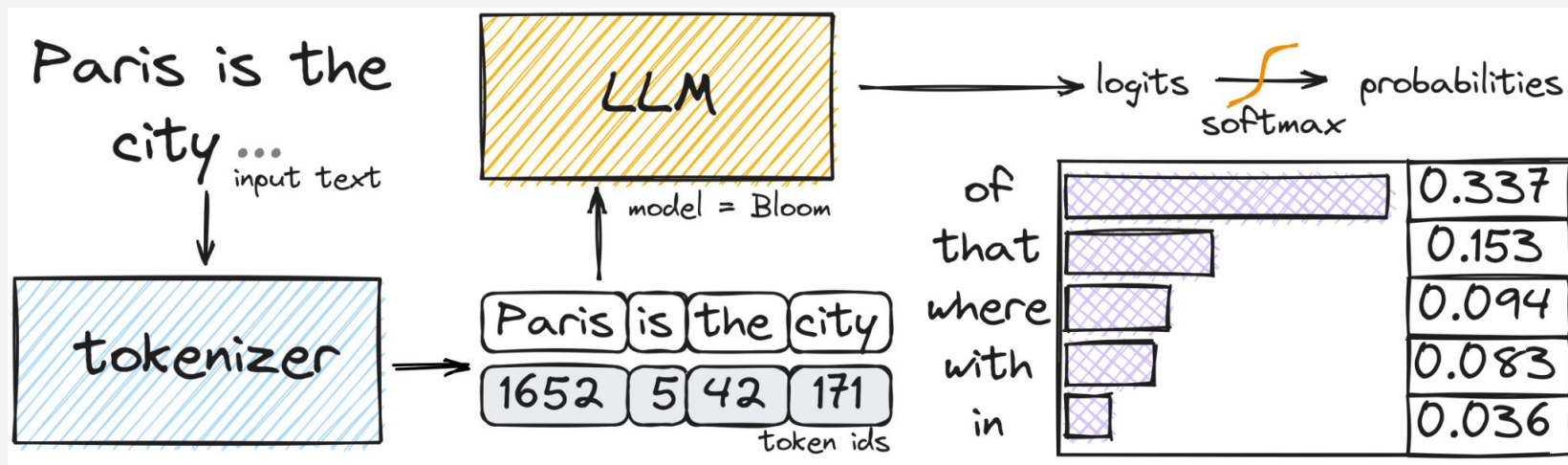


How do LLMs generate text?



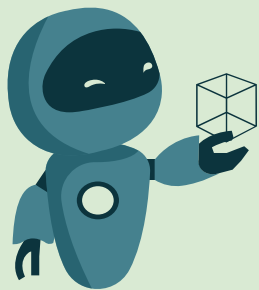
Resource: <https://www.linkedin.com/pulse/how-exactly-llm-generates-text-ivan-reznikov/>

How the Encoder works



Strategies to select the token

Method	Description	Diversity	Computation Efficiency	Quality of Output
Greedy Search	Selects the most probable token at each step.	Low	High	May lack diversity
Beam Search	Considers top-K probable tokens in parallel.	Moderate	Medium-High	Improved diversity
Random Sampling	Randomly selects tokens based on probabilities.	High	Medium	May lack coherence
Random Sampling with Temperature	Adds randomness based on temperature.	High	Medium	More controlled randomness
Top-K Sampling	Selects top-K probable tokens randomly.	High	Medium	Improved diversity
Nucleus Sampling	Selects tokens until cumulative probability $\leq p$.	High	Medium-High	Improved coherence



03

Design Process of Building an AI Agent

Step 1: Define what you want to do

What is the Problem?

Job seekers spend 2–4 hours manually tailoring résumés for each application. They need to rewrite bullets, adjust keywords, and ensure the design looks professional but they lack writing skills and design sense.



Who: Job seekers



Pain: Wastes time, lacks skills



Desired outcome: Faster, better résumés

Step 1: Define what you want to do

What is the scope of the project?

1.  **Input:** Users upload a resume (PDF) + Job title
2.  **Output:** Users upload a resume (PDF) + Job title
3.  **Out of scope:** Apply to jobs automatically or generate résumés from scratch

Step 2: Decide on architecture

Single Agent

- The workflow is sequential
- One 'role' suffices
- You want simplicity and low latency

Multi-Agent

- Tasks need specialized roles
- Agents need to collaborate
- Complexity is worth the trade-off

Human Machine

- Decisions are high-stakes
- The agent needs approval before acting
- You're validating before full automation



Step 2: Decide on architecture

When to upgrade to Multi agent?

If you wanted to add:

1. **Research Agent** that scrapes job postings and extracts keywords
2. **Formatting Agent** that converts the enhanced text to a Word document
3. **QA Agent** that checks for typos and ATS compliance



Step 3: Decide the components



1. Models

The core intelligence for reasoning and decision-making. Usually LLMs.



2. Tools

External systems and APIs that agents use to perform actions in the real world.



3. Knowledge

Systems for storing and retrieving information, including past interactions and learned data.



4. Interfaces

How agents interact with the outside world, such as web apps or APIs.



5. Orchestration

Manages tasks, steps, and workflows to guide the agent's actions.



6. Instruction

Clear instructions and context that guide the agent's focus.



7. Guardrails

Safety mechanisms to ensure proper behavior.

Step 3a: How to design tools?

Key Considerations:

- ✓  What tasks **can't** the LLM do alone?
- ✓  What **specific actions** does your agent need?
- ✓  Keep it **focused**: too many tools confuse the agent
- ✓  Make tools **atomic**: Each does one thing well

Step 3a: How to design tools?

Our Tools:



-  `parse_resume`: Extracts text from PDF resumes
-  `enhance_resume`: Improves content with AI suggestions
-  `vision_style_analyzer`: Analyzes visual formatting using GPT-4o



Step 3a: How to design tools?

Key principles when coding the tools:

- **Name:** Verb + noun (parse_resume, not resume_parser)
- **Docstring:** What it does + when to use it (the agent reads this!)
- **Inputs:** Typed parameters (pdf_path: str, not just 'a file')
- **Outputs:** JSON-serializable (dict, str, list—no raw objects)
- **Failure modes:** What can go wrong? (corrupted PDF, missing file)
- **Make tools atomic. Each does one thing well.** Don't combine parsing and enhancing as it's harder to debug and reuse.



Step 3b: How to decide on a model?

Key Considerations:

- ✓ 💡 **Capability:** Does it understand complex instructions?
- ✓ ⚡ **Speed:** How fast does it respond?
- ✓ 💰 **Cost:** Token pricing for your use case
- ✓ 🧠 **Context Window:** How much information can it process?
- ✓ 🛠️ **Function Calling:** Can it use tools effectively?



Step 3b: How to decide on a model?

Model	Capability	Speed	Cost (Input/Output per 1M tokens)	Context Window	Function Calling	Recommendation
GPT-4o	Excellent reasoning, reliable tool orchestration	Fast (~2-4s)	\$2.50 / \$10.00	128k	Native, very reliable	Best balance
Claude Sonnet 4.5	Superior reasoning, best-in-class tool use	Fast (~2-4s)	\$3.00 / \$15.00	200k	Native, excellent	Best for complex
Gemini 1.5 Pro	Strong reasoning, good orchestration, multimodal	Medium (~4-6s per call)	\$1.25 / \$5.00	2M tokens (!!)	Native (via function declarations)	Budget option Huge context, cheaper
Llama 3.3 70B (via Groq/Together)	Decent reasoning, open-source	Very Fast (~1-3s on Groq)	\$0.60 / \$0.60 (Together)	128k	Via JSON mode (manual parsing)	Open-source option. Self-host or low-cost API



GPT Models

GPT refers to a suite of powerful language models developed by OpenAI, such as GPT4. These models can perform a wide variety of tasks, including text generation, conversation, translation, and image processing (in the case of GPT-4 with Vision).

Through OpenAI's API, users can access these models directly via OpenAI's platform, bypassing the need for third-party integrations like Azure. With an OpenAI API key, you have direct access to the latest models OpenAI offers.

While our focus for this training is on OpenAI's models, which are the most widely used and well-documented, the principles for interacting with other large language models (LLMs) like Meta's LLaMA or Anthropic's Claude are largely similar.

LLM Leaderboard

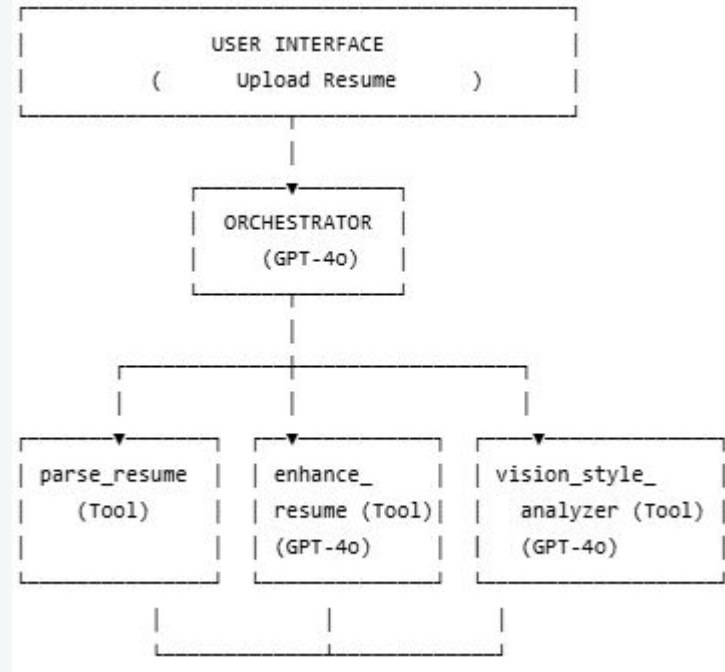


Resource: <https://www.vellum.ai/llm-leaderboard>

Step 3c: Orchestration

It is the logic that manages:

- Which tools to call and when
- The sequence/order of operations
- Error handling and retries
- Combining results into final output



Step 3d: Instruction/Prompt

Orchestrator prompt = 'Manager'

- High-level workflow
- Decision logic (if/then)
- Error recovery
- No domain expertise

Tool prompts = 'Specialists'

- Deep task expertise
- Detailed constraints
- Quality criteria
- No workflow knowledge



Step 3d: Instruction/Prompt

Three key parts:

1. **Identity/Role:** 'You are a [role].'
2. **Tools:** List available tools + when to use each.
3. **Workflow:** Step-by-step logic ('First do X, then Y').

Step 3d: Instruction/Prompt

You are the Resume Enhancement Agent.

You only “communicate” by calling one of these three functions:

1. `parse_resume(pdf_path: str, job_spec: str) → {text, job_spec}`
 - Always call this first to extract raw text against the spec.
2. `enhance_resume(text: str, job_spec: str) → {enhanced}`
 - Use this on the parsed text to rewrite/improve the résumé.
3. `vision_style_analyzer(pdf_path: str) → {style_score, template_type, suggestions}`
 - Use this when the user asks for a style critique or template feedback.

If the user asks to critique the visual design or template of the resume, call `vision_style_analyzer`.

If the user asks to enhance content, follow `parse_resume → enhance_resume` as before.



Step 3e: Interface

Common Interface Types:



Chat/Conversational → Web chat, Slack, Discord



Web Applications → REST APIs, webhooks



Voice Interfaces → Voice commands, speech responses



Mobile/Desktop Apps → Native applications



Step 3e: Interface

AI-Powered Résumé Enricher + Style Critic

Upload Résumé (PDF)


Drop File Here
- or -
Click to Upload

Job Specification

e.g. Senior Software Engineer

Enhance Résumé

Critique Style

Result

Step 4: Implementation



Langchain

- Most mature and widely adopted
- Extensive tool ecosystem
- Great documentation



Google ADK

- Optimized for Google's Gemini models
- Newer, growing ecosystem



n8n

- Visual, no-code workflow builder
- Great for non-technical teams
- Limited customization



Step 4: Implementation

You have a design. Now: How do you build it? Here's a four-phase plan:

Phase 1: Build Tools in Isolation

Write each tool as a standalone Python function. Test it with sample inputs. Don't integrate with the agent yet. Why? Because if `parse_resume` doesn't work on its own, it won't work in an agent.

Phase 2: Wire the Agent (Minimal Prompt)

Create the agent with a simple system prompt: 'You have three tools. Call them as needed.' Test: Can the agent call the tools? Don't worry about perfect behavior yet—just verify the plumbing works.

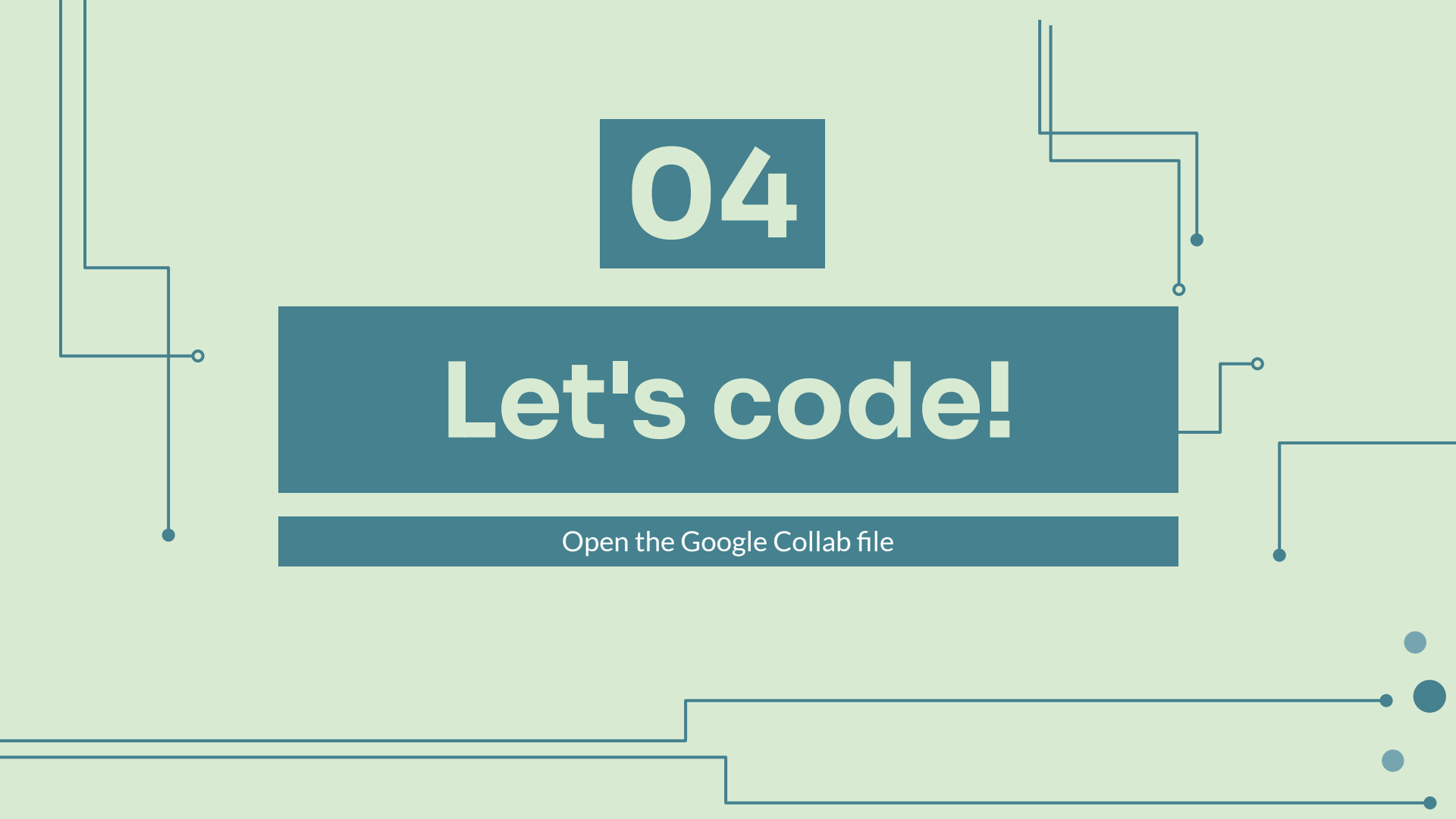
Phase 3: Refine the Prompt

Now iterate on the system message. Add workflow rules: 'Always parse before enhancing.' Test with diverse queries. Debug when the agent calls the wrong tool. Refine until behavior is reliable.

Phase 4: Add Interface & Guardrails

Wrap the agent in Gradio. Add error handling, input validation, rate limits. Test edge cases: corrupted PDFs, empty inputs, huge files. Polish the UX."



The background features a light green color with dark blue circuit-like lines. These lines are composed of horizontal and vertical segments connected by small circles, resembling a printed circuit board (PCB) layout. Some lines end in open circles, while others end in solid dots. The lines are distributed across the slide, with a notable cluster in the top right and another in the bottom right.

04

Let's code!

Open the Google Collab file

Steps to get Open AI API Key

Step 1: Sign in to <https://platform.openai.com/api-keys>



Welcome to OpenAI Platform

Create an organization to generate
API keys and start building.

Organization name

Personal

What best describes you?

Student

Create organization



Invite your team

Members can make standard API requests
and read basic organizational data.

Emails

Owner ↕

name@company.com

Continue

I'll invite my team later

Steps to get Open AI API Key

Step 2: Click on "Generate API Key" / "Create new secret key".



Make your first API call

Create a project and generate a key to make your first API call.

API key name

Project name

API keys belong to projects to help you manage usage limits, team access, and data security.

Generate API Key

I'll do this later

Personal

Default project

Dashboard Docs API reference

Create

Chat

Audio

Images

Assistants

Manage

Usage

API keys

Logs

Storage

Batches

Optimize

Evaluation

Fine-tuning

API keys

+ Create new secret key

You have permission to view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED	CREATED BY	PERMISSIONS
DAC AI Agent	sk-...qasA	Never	Juliana Charisse Ramos	All

Steps to get Open AI API Key

Step 3: Paste API key here in the Secrets tab with the Name as OPENAI_API_KEY

Secrets

Configure your code by storing environment variables, file paths, or keys. Values stored here are private, visible only to you and the notebooks that you select.

Secret name cannot contain spaces.

Notebook access	Name	Value	Actions
☒	OPENAI_		

[+ Add new secret](#)

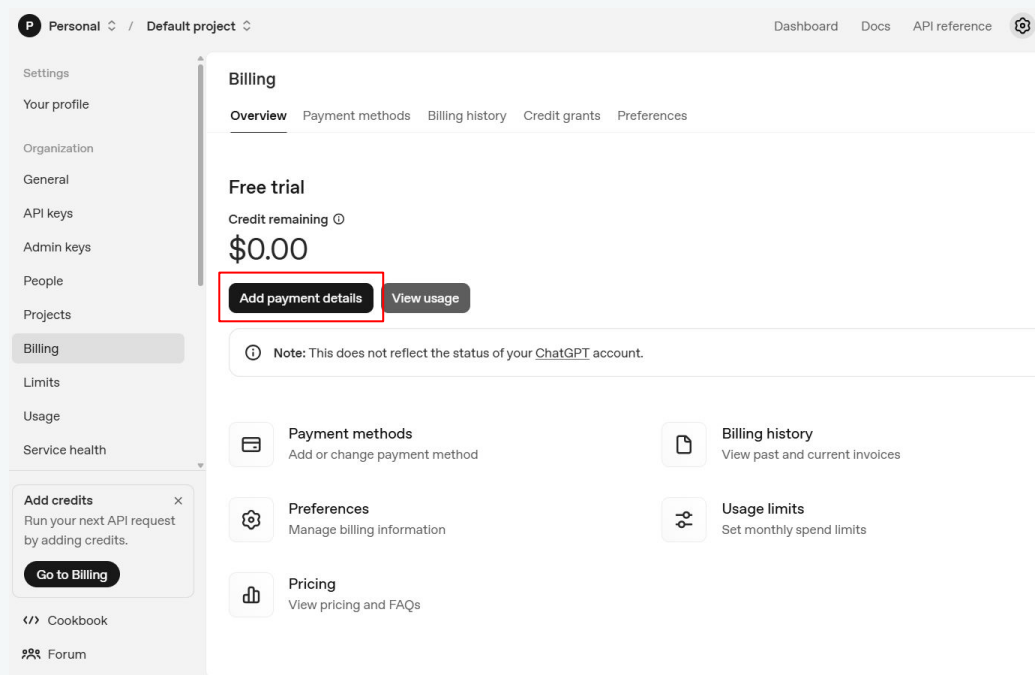
Gemini API keys ▾

Access your secret keys in Python via:

```
from google.colab import userdata
userdata.get('secretName')
```

Steps to get Open AI API Key

Step 4: Click on "Add payment details"



The screenshot shows the OpenAI Billing page. The left sidebar contains a navigation menu with the following items: Settings, Your profile, Organization, General, API keys, Admin keys, People, Projects, Billing (highlighted), Limits, Usage, and Service health. At the bottom of the sidebar are links for Cookbook and Forum. The main content area is titled 'Billing' and has tabs for Overview, Payment methods, Billing history, Credit grants, and Preferences. The 'Overview' tab is active, showing a 'Free trial' status with 'Credit remaining' of '\$0.00'. A red box highlights the 'Add payment details' button, with a 'View usage' button next to it. Below this, a note states: 'Note: This does not reflect the status of your ChatGPT account.' At the bottom, there are five cards: 'Payment methods' (Add or change payment method), 'Billing history' (View past and current invoices), 'Preferences' (Manage billing information), 'Usage limits' (Set monthly spend limits), and 'Pricing' (View pricing and FAQs).

Personal / Default project

Dashboard Docs API reference

Settings

Your profile

Organization

General

API keys

Admin keys

People

Projects

Billing

Limits

Usage

Service health

Add credits x
Run your next API request by adding credits.
[Go to Billing](#)

Cookbook

Forum

Billing

Overview Payment methods Billing history Credit grants Preferences

Free trial

Credit remaining ⓘ

\$0.00

[Add payment details](#) [View usage](#)

ⓘ Note: This does not reflect the status of your ChatGPT account.

Payment methods
Add or change payment method

Billing history
View past and current invoices

Preferences
Manage billing information

Usage limits
Set monthly spend limits

Pricing
View pricing and FAQs

Steps to get Open AI API Key

Step 5: After putting in your card details, add \$5 then Continue

Configure payment

Initial credit purchase

\$ 5

Enter an amount between \$5 and \$100

Model pricing [C2](#)

Would you like to set up automatic recharge?

☐ Yes, automatically recharge my card when my credit balance falls below a threshold

When credit balance goes below

\$

Enter an amount between \$5 and \$95

Bring credit balance back up to

\$

Enter an amount between \$10 and \$100

Limit the amount of automatic recharge per month

\$

Enter an amount between \$10 and \$100. Leave this field empty for no recharge limit.

Back

Continue

Billing

[Overview](#) [Payment methods](#) [Billing history](#)

Free trial

Credit remaining ⓘ

\$5.00

Add payment details

View usage

Steps to get Open AI API Key

You should have \$5 credits now!

Billing

Overview

Payment methods

Billing history

Free trial

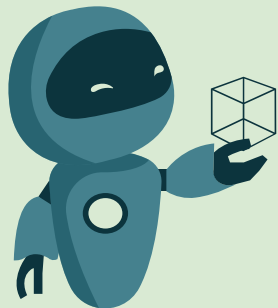
Credit remaining ⓘ

\$5.00

[Add payment details](#)

[View usage](#)





Extra Resources

Helpful Resources

That I personally watched and found useful

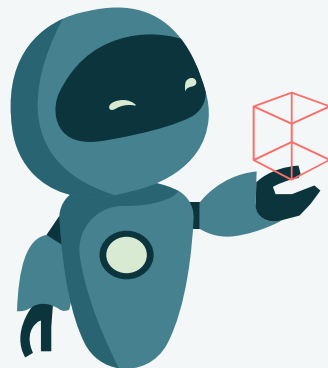
Video Title and Link	Length
AI Agents Fundamentals (Link)	21 Min
Building AI Agents (Link)	44 Min
LangChain Explained (Link)	12 Min
LangChain Masterclass (Link)	3 Hr 17 Min

AI Agent Repositories

<https://github.com/kyrolabs/awesome-langchain>

<https://github.com/Arindam200/awesome-ai-apps>

<https://github.com/ashishpatel26/500-AI-Agents-Projects>



Google ADK Resources

I lowkey prefer this framework over Langchain..

Video Title and Link	Duration	My Thoughts
Google Agent Development Kit Masterclass (Link)	3 Hrs	Helpful if you want to learn Google ADK. Has strong connection to Google tools. Takes you from Beginner to Pro. Engaging speaker!
Google ADK Playlist (Link)	3.5 Hrs Total	The project goes through a Marketing Campaign Assistant - Google ADK Tutorial Project (Repo link : I suggest you run it and explore!). Goes in depth into multi-agents



Rakesh Gohel
@rakeshgohel01

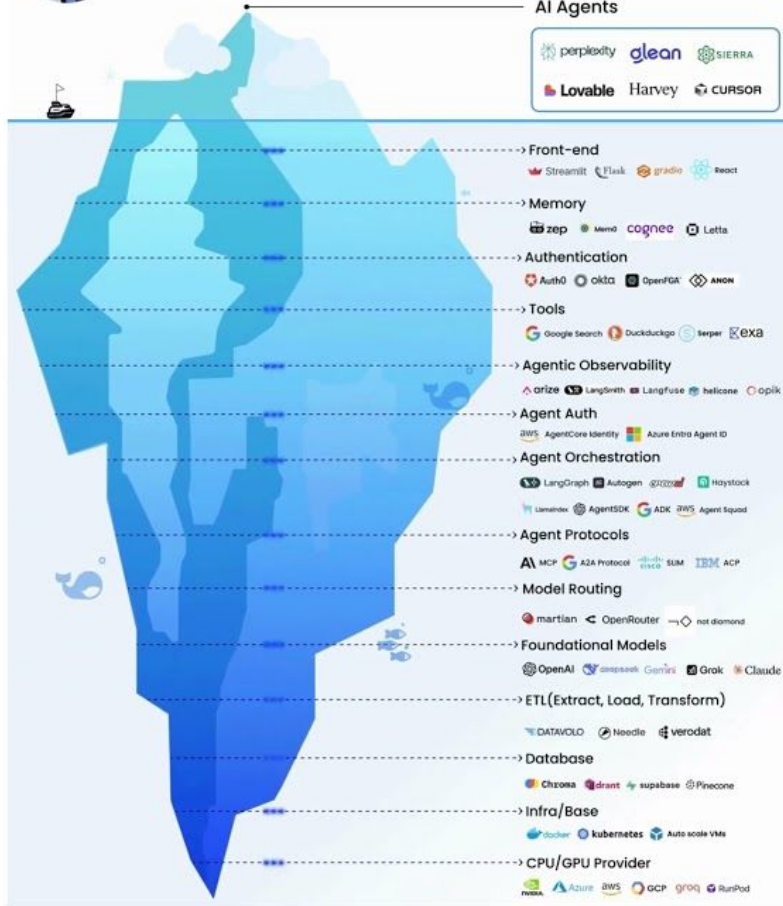


AI Agents are

90% Software Engineering

10% AI

AI Agents



Popular AI Tools Everyone Knows



ChatGPT



Google Gemini



Microsoft Copilot



Grammarly AI



Canva Magic Write



Notion AI

AI for content creation

ChatGPT for emails, summaries, research

AI social media schedulers

Beginner AI Use Cases & Skills

Prompt templates

Resume building with AI

AI meme generators

Intermediate Tools & Concepts

Prompt Engineering basics

ElevenLabs, D-ID, Pictory

No-code AI workflow builders

AI automations (Zapier, Make.com)

AI avatars & video creation

Speech-to-text with Whisper

OpenAI API / Anthropic API

Fine-tuning LLMs

Vector databases (Pinecone, Chroma, Weaviate)

Developer-Level AI Concepts

LangChain & LangGraph

Embeddings

Tokenization

Advanced Agentic Systems

Autonomous AI agents

Tool use via ReAct & LangGraph

Multi-agent collaboration

AgentOps / Agent Workflow orchestration

Memory management

Function calling

LLM Monitoring & Evaluation

Self-healing AI pipelines

Custom embeddings & reranking

LLMOps & Deep Customization

Retrieval-Augmented Generation (RAG)

Synthetic data generation

Model quantization & distillation

Theoretical AI & Alignment

Chain-of-thought reasoning

Emergent behaviors in LLMs

Instruct tuning vs RLHF

AI Alignment & safety

Reward modeling

Causal inference in ML

AGI design principles

Self-improving AI

Autonomous economic agents

Frontiers of AI / AGI

Neural-symbolic systems

Artificial consciousness

AI x Neuroscience fusion